TEORÍA DE LA COMPUTACIÓN – INFO1148

Proyecto: Diseño e implementación de un analizador sintáctico

Antecedente	Información
Modalidad	Proyecto grupal
Tema central	Análisis sintáctico y gramáticas libres de contexto
Puntaje	100 puntos
Lenguaje de implementación	Python
Entrega	Informe + (código fuente y resultados)

1. Contexto del proyecto

Los lenguajes de programación y los lenguajes de dominio específico requieren mecanismos capaces de verificar si una secuencia de símbolos cumple una estructura formal. En este proyecto, cada grupo diseñará un pequeño lenguaje de dominio específico (DSL) y desarrollará un analizador sintáctico capaz de reconocer programas válidos, detectar errores y construir una representación sintáctica de la entrada.

2. Objetivo general

Diseñar e implementar un analizador sintáctico para un mini lenguaje, aplicando gramáticas libres de contexto, conjuntos FIRST y FOLLOW, análisis LL(1), construcción de árboles sintácticos y detección de errores.

3. Elección del dominio: componente creativo

Cada grupo deberá escoger un dominio de preferencia para su lenguaje. Ejemplos sugeridos:

- automatización de una casa inteligente o sistema IoT
- comandos para un robot o dron
- reglas de comportamiento de un videojuego
- automatización agrícola o monitoreo ambiental
- reglas simples de ciberseguridad
- lenguaje de consultas o filtrado de datos
- control de procesos o máquinas

Importante: aunque el dominio cambie, todos los grupos deberán cumplir los mismos requisitos sintácticos mínimos indicados en este enunciado del proyecto.

4. Ejemplo orientativo

Un grupo que elija automatización IoT podría definir sentencias como:

WHEN temperatura > 28 THEN encender ventilador;

WHEN movimiento == TRUE THEN activar alarma;

WHEN temperatura > 28 AND humedad < 40 THEN encender ventilador;

Este ejemplo es únicamente orientativo. Cada grupo debe adaptar el vocabulario, tokens, gramática y ejemplos propios del dominio.

5. Requisitos mínimos del lenguaje

- Al menos 4 palabras reservadas.
- Identificadores.
- Valores numéricos y/o booleanos.
- Al menos 3 operadores (relacionales, lógicos o propios del dominio).
- Una estructura de sentencia principal.
- Posibilidad de escribir más de una sentencia o regla.
- Al menos una construcción compuesta, por ejemplo AND, OR, parámetros, bloque o agrupación.

6. Fases obligatorias del proyecto

6.1 Definición formal del lenguaje

1. Describir el propósito del lenguaje y el dominio escogido.
2. Definir el alfabeto relevante y las categorías de tokens.
3. Identificar palabras reservadas, identificadores, operadores, valores y separadores.
4. Presentar al menos 5 ejemplos de sentencias válidas y 5 inválidas.

6.2 Diseño de la gramática libre de contexto

Construir una GLC que represente el lenguaje diseñado. La gramática debe identificar claramente símbolo inicial, terminales, no terminales y producciones.

Ejemplo orientativo de estructura:

<programa> → <regla> <programa> | <regla>

<regla> → WHEN <condicion> THEN <accion> ;

<condicion> → <comparacion> <condicion2>

<condicion2> → AND <comparacion> <condicion2> | OR <comparacion> <condicion2> | ε

6.3 Análisis de la gramática

5. Identificar terminales y no terminales.
6. Indicar el símbolo inicial.
7. Analizar si existe recursión izquierda.
8. Analizar si se requiere factorización por la izquierda.
9. Discutir posibles ambigüedades y, si corresponde, modificar la gramática.
10. Calcular FIRST para los no terminales relevantes.
11. Calcular FOLLOW para los no terminales relevantes.

6.4 Construcción de la tabla LL(1)

A partir de FIRST y FOLLOW, construir la tabla predictiva LL(1). El grupo deberá indicar si la gramática es LL(1). Si aparecen conflictos, deberá explicar su causa y modificar la gramática hasta obtener una versión utilizable por el analizador, siempre que sea posible.

6.5 Implementación del analizador sintáctico

Implementar un analizador sintáctico predictivo, preferentemente mediante descenso recursivo o una tabla LL(1). El programa debe recibir una secuencia de tokens y determinar si la entrada es sintácticamente válida.

- Aceptar correctamente sentencias válidas.
- Rechazar sentencias que no respeten la gramática.
- No depender de una biblioteca que construya automáticamente todo el parser.
- Organizar el código en funciones o módulos comprensibles.

6.6 Árbol sintáctico o AST

Para al menos una entrada válida, el sistema deberá construir una representación estructurada, ya sea árbol sintáctico o AST. Puede representarse mediante texto jerárquico, Graphviz, NetworkX u otra herramienta equivalente.

6.7 Manejo de errores sintácticos

El programa no debe limitarse a responder “error”. Debe entregar mensajes útiles, por ejemplo:

Error sintáctico: se esperaba un valor después del operador >.

Error sintáctico: se esperaba la palabra reservada THEN.

Error sintáctico: símbolo inesperado cerca de la posición 5.

6.8 Casos de prueba

Cada grupo deberá documentar, como mínimo:

- 10 entradas sintácticamente válidas.
- 10 entradas sintácticamente inválidas.

- 5 casos límite.

Entrada	Resultado esperado	Resultado obtenido	Observación

6.9 Extensión creativa obligatoria

Cada grupo deberá incorporar una extensión que aumente la expresividad de su lenguaje. Ejemplos:

- paréntesis o agrupación
- operador NOT
- múltiples acciones
- estructuras IF–ELSE
- acciones con parámetros
- expresiones aritméticas
- comentarios
- bloques anidados
- una construcción propia del dominio elegido

La extensión deberá reflejarse en la gramática, FIRST/FOLLOW cuando corresponda, tabla LL(1), implementación y casos de prueba.

7. Componente léxico

El foco del proyecto es el análisis sintáctico. Por ello, el grupo puede implementar un analizador léxico sencillo o utilizar una función entregada por el docente que transforme la entrada en tokens. El trabajo evaluado se concentrará principalmente en la gramática y el parser.

8. Restricciones

- No se permite utilizar un generador completo de parsers que resuelva automáticamente la gramática y el análisis sintáctico.
- Se permiten bibliotecas auxiliares para estructuras de datos, visualización, lectura de archivos y generación gráfica del árbol.
- El grupo debe ser capaz de explicar la relación entre la gramática y cada parte del código implementado.
- Si se utiliza IA generativa como apoyo, el grupo sigue siendo responsable de comprender, justificar y defender toda la solución presentada.

9. Entregables

12. Informe en PDF: dominio, lenguaje, gramática, análisis, FIRST/FOLLOW, tabla LL(1), pruebas, árboles y conclusiones.
13. Código fuente completo documentado (con link en anexo que lleve a un GitHub)
14. README con instrucciones de ejecución, dependencias, formato de entrada y ejemplos. (También en el GitHub)
15. Archivos auxiliares necesarios. (Incluier en el GitHub)
16. Demostración funcional del Proyecto (capturas de pantallas en el informe sección de anexos)

10. Sugerencia para organización del informe

1. Introducción y dominio seleccionado
2. Especificación del lenguaje
3. Gramática libre de contexto
4. Análisis de la gramática
5. FIRST y FOLLOW
6. Tabla LL(1)
7. Diseño e implementación del parser
8. Árbol sintáctico / AST
9. Manejo de errores
10. Casos de prueba
11. Conclusiones
12. Anexos

11. Distribución de puntaje

Componente	Puntaje
Definición del lenguaje y dominio	10
Gramática libre de contexto	15
Análisis de gramática + FIRST/FOLLOW	20
Tabla LL(1)	10

Implementación del parser	20
Árbol/AST y manejo de errores	10
Casos de prueba y extensión creativa	10
Informe, README	5
TOTAL	**100**